

UNIT 03

ALGORITHMIC THINKING AND PROBLEM SOLVING



Student Learning Outcomes:

After the completion of this unit students will be able to:

- Define and identify a problem and differentiate between simple and complex problems.
- Solve any problem by following the 6 steps problem solving guide.
- Analyze different steps to breakdown a problem.
- List benefits of algorithmic thinking.
- Recognize that algorithms are sequence of precisely described instructions.
- Define conditional/selection statements that decide whether certain instructions should run or not?
- Identify loops and analyze how they allow instructions to be repeated.
- Analyze ways to solve a problem by using a combination of sequence, selection, and repetition.

Introduction to the Unit

In this chapter we are going to learn how to solve problems in a well manner, which is super important for everything we do, like at school and even when we grow up and have jobs. This chapter also introduces the fundamental principles of algorithmic thinking, highlighting its role as a systematic approach to breaking down problems into manageable components and planning step-by-step strategies for solution.



3.1 PROBLEM

A problem is something (event, task, situation) that needs to be solved. It is like a puzzle or question for which a solution must be found. Problems can be big or small, and can be from the real-world or pointed out by someone. They can be anything from real life situations to math problems, for instance.

1. You are walking in the park, and you see a lost puppy, what would you do to help the puppy find its way home.
2. Your favorite toy is broken, what are some ways you can try to fix it?
3. If Aslam has 3 bags of coins, and each bag has 5 coins, how many coins does Aslam have in total?



Figure 3.1 Example of Problems

3.2 SIMPLE AND COMPLEX PROBLEM

3.2.1 Simple Problems

Simple problems are like puzzles that are easy to solve. They usually have a clear question with an answer that you can find using basic steps.

For example,

Bake a cake or cook instant noodles for lunch.

- Simple problems can be solved by simply following the recipes.
- No expertise is required to solve simple problems.

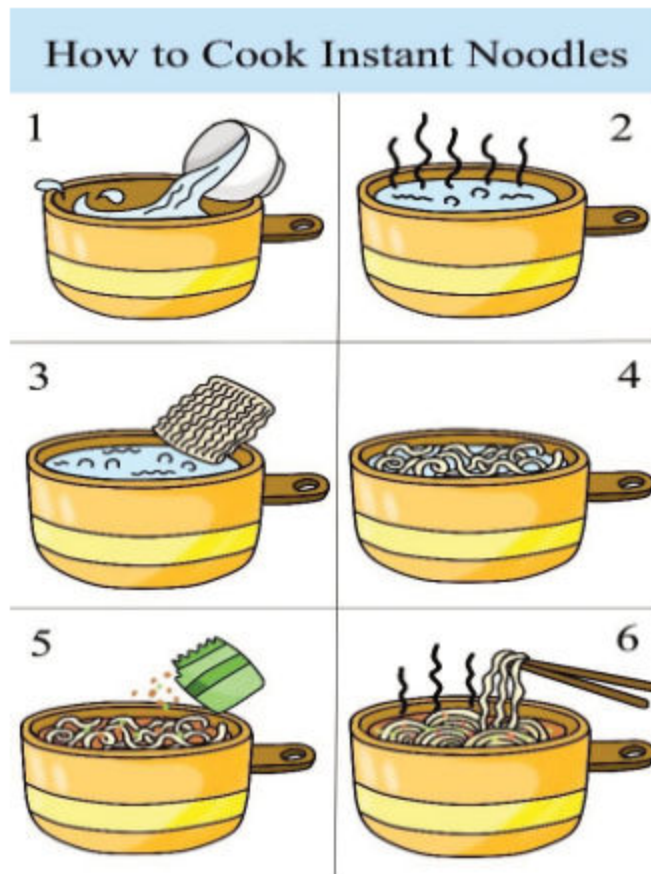


Figure 3.2 Steps to Cook Instant Noodles

3.2.2 Complex Problems

Complex problems are like larger puzzles that need more thinking. They might have many steps or need different ways to solve them.

For example,

- Calculating the Least Common Multiple (LCM)
- Determining if the number is prime or not.
- A book costs 500 Rs, more than twice the cost of a pen. If the total cost of the book and the pen is 800 Rs, how much does each item cost?

These types of problems can be solved by breaking them into many simple problems.

2	10 , 18 , 20
2	5 , 9 , 10
3	5 , 9 , 5
3	5 , 3 , 5
5	5 , 1 , 5
	1 , 1 , 1

$$\text{LCM} = 2 \times 2 \times 3 \times 3 \times 5$$

$$\text{LCM} = 180$$

Figure 3.3 Calculating the LCM of 10,18 and 20

ACTIVITY

Teacher will present different problems and ask students if each one of the problems is simple or complex.

3.3 PROBLEM SOLVING

We solve various problems every day and every problem is unique in nature. Some problems are difficult and require more attention to find the solution. Problem solving is a process of identifying and finding the most effective solution to a problem. The skill of problem solving can be mastered by following a systematic and well-organized approach as shown in figure 3.4.

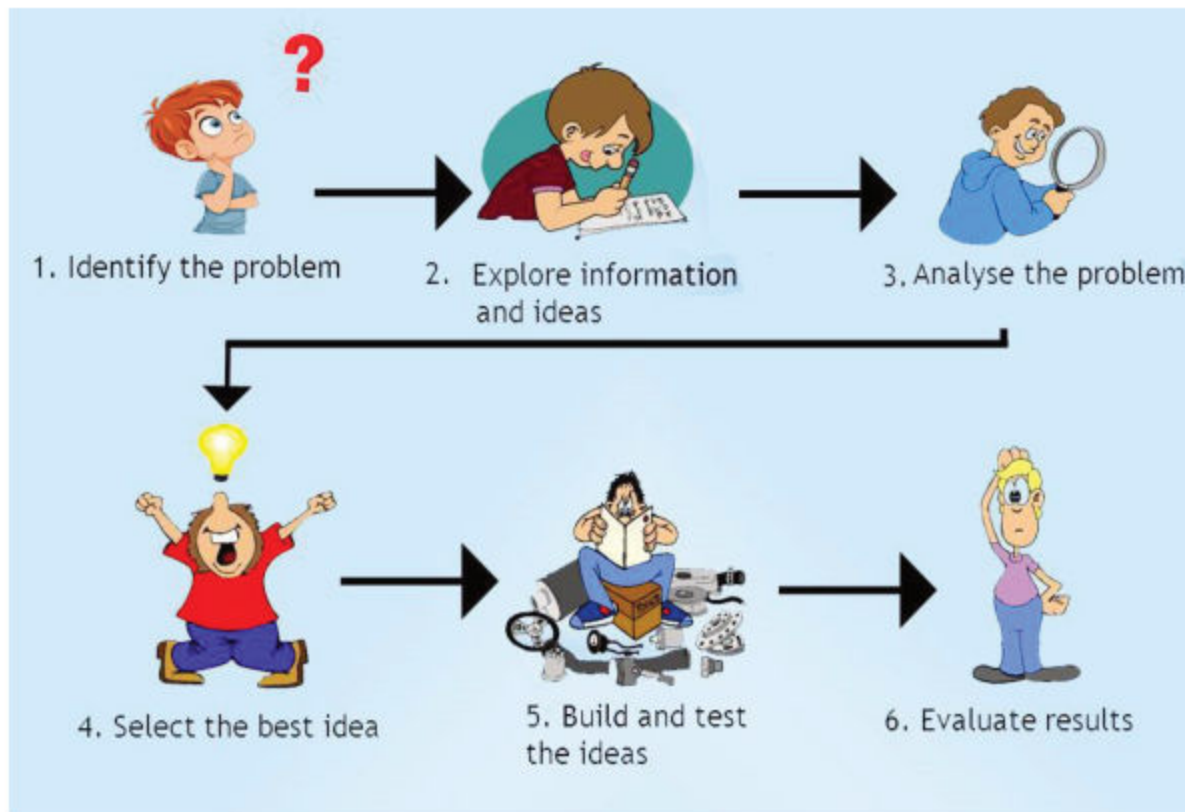


Figure 3.4 The six-step problem solving approach

3.3.1 Six step guide to help you solve problems

(I) Step 1: Identify the problem

- State what the problem is. For example: “Manahil wants to buy a birthday gift for her friend Moomal”.
- Be clear about when, where, and why it is problem, for example: “Manahil wants to buy a birthday gift that Moomal will like, and which will make their friendship stronger.”



Figure 3.5 Birthday gift surprise

(II) Step 2: Explore information and ideas

- Write down all the ideas you can think of, even if they seem wrong or not good.
- Use your imagination to think of some cool ideas you would not have thought of before.

In this case, Manahil can think of the following possible solutions.

1. A book that her friend mentioned that she wants.
2. A board game, they can play together.
3. A gift card to their favorite store.
4. A personalized photo album of their memories together.



Figure 3.6 Birthday gift ideas, a gift card, a book, a board game

(III) Step 3: Analyze the problem and evaluate alternatives

- We look for ideas we don't like as much and say bye-bye to them.
- We put the ideas we like the most on a special list.
- We think about good and not so good things about each idea to decide which one is best.

For example, Manahil came to the following conclusion for each possible solution that she can think of.

1. The book might be a good choice because her friend loves reading but buying a book can be expensive.
2. The board game could be fun for them to enjoy together but what if she already has the game you buy.
3. A gift card gives her friend the option to choose what she wants to buy but Manahil does not know her friend's likes or dislikes.
4. The photo album would be a sentimental and thoughtful gift.

(IV) Step 4: Select the best idea

- Say who is going to do something about it, like, "Manahil will prepare a personalized photo album for her friend".
- Say how you are going to fix it. For example, "Manahil will collect photos of her friend and create a beautiful photo album".
- Say when you do it, "Manahil will start collecting the photos from today and must complete the album before her friend's birthday."

(V) Step 5: Build and test the ideas

- Do what you decided, "Manahil starts collecting the photos of her friend from various sources such as friends and family and creates a beautiful photo album."



Figure 3.7 Personalized photo album

(VI) Step 6: Evaluate results

- Think if the plan worked, Like, “Did Manahil’s friend loves the gift and feels touched by the memories captured in the album.?”
- Decide if you need to change the plan or make a new one to fix the problem.
- If you are not happy with what happened, note this for the future and start again from step 2, pick a new plan, or change the old one etc.

Example # 1: Dealing with bullying at School

- **Identify and define the problem:** Aslam is being bullied at school and needs to find a way to handle the situation.



Figure 3.8 Student being bullied

- **Explore information and ideas**
 - Tell a grown up you trust such as a teacher, counselor, guardian or parent.
 - Face the bully with courage and tell them to stop bothering you
 - Ask friends or classmates for support and help.
 - Be part of groups at school that stop bullying.
- **Analyze the problem and evaluate alternatives**
 - Telling a grown up for help and support can stop the bully.
 - If you face the bully with courage, they might stop bullying you.
 - Your friends can help you feel better and strong, but your friends may also start bullying you.
 - Becoming parts of anti-bullying groups may help you stop bullying but it also diverts your attention from studies.
- **Select the best idea:** Aslam decides to talk to his teacher about bullying.
- **Test the idea:** He met with his teacher, explains the situation, and asks for help.



Figure 3.9 Student seeking help from teacher

- **Evaluate results:** Aslam receives support from the school, the bullying stops, and he feels safer and happier at school.



ACTIVITY

Example of real-life problem

Watch and Learn: Teacher must give an example of a real-life problem he/she has faced and discuss how he/she has solved it.

Hands-on Practice: After teachers' demonstration, each student should think of a real-life problem he/she has faced and follow the six steps problem solving guidelines to prepare a solution of that problem

3.4 PROBLEM DECOMPOSITION

Sometimes the problem is too large to solve all at once. Therefore, we decompose or divide problems into smaller sub problems. This technique is known as “Problem decomposition”. The decomposition of any problem can be done by dividing the problems into facts, input, outputs and instructions.

3.4.1 Facts

Facts are things we know that are true or real. They are like the building blocks of problem-solving process. For example, if we are baking bread, some facts could be that we need flour, yeast, oil, warm water, and salt.

3.4.2 Input

Inputs are the things we start with, like the ingredients for our bread (if that is the problem).

3.4.3 Output

Outputs are the cool things we get at the end, like fresh yummy bread! They are what we make or get after we put everything together.

3.4.4 Instructions

We combine facts, inputs, and outputs to create a list of steps (**instructions**) that guide us in solving the problem. There could be two different types of instructions:

(I) Specific instructions

Specific instructions give clear, detailed steps on how to do something.

For example, imagine you are baking a cake, a specific instruction would be

“Bake the cake for 40 minutes at 200 degrees Celsius”

(II) Non-specific instructions

Non-specific instructions are more general and less detailed. In the cake baking scenario, a non-specific instruction would be

“Mix all the ingredients then cover it and let it sit”

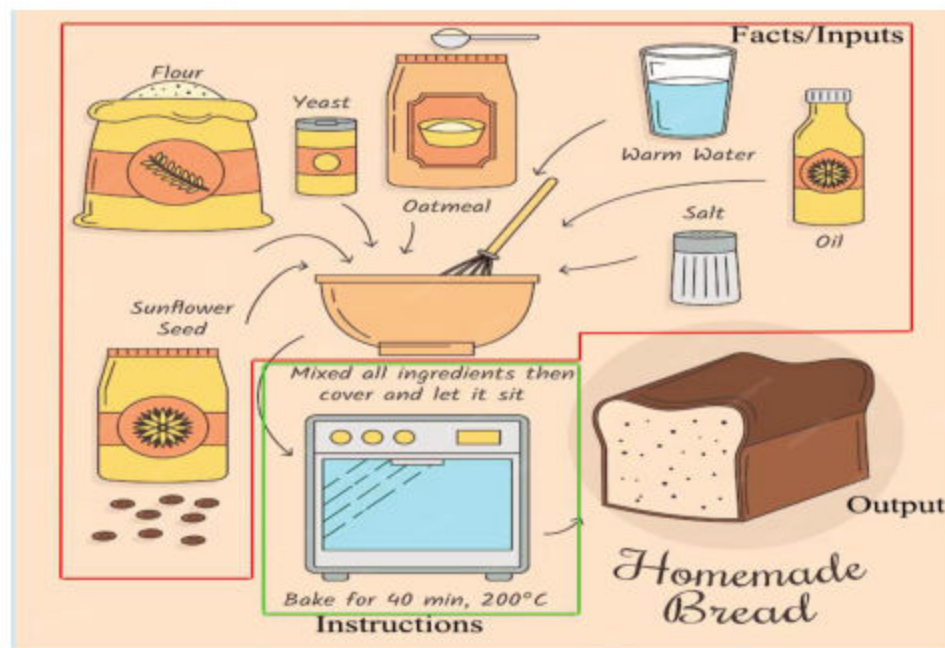


Figure 3.10 Baking bread ingredients and tools

Example # 1: Building a LEGO castle

- **Facts:** we need LEGO bricks of different colors and sizes and may be some special pieces like windows and doors.



Figure 3.11 LEGO blocks of different colors and shapes

- **Input:** A blank canvas or an empty space where we want to build the castle.
- **Output:** How we like our castle to look like, say we want a tall castle with towers, windows, a big gate, and maybe even a drawbridge.
- **Pulling it together (Instructions)**
 - We have LEGO bricks of different colors and sizes.
 - We start with an empty space or a blank canvas.
 - We want a castle with towers, windows, a gate, and a drawbridge.



Figure 3.12 LEGO Castle

ACTIVITY

Watch and Learn: Teacher must give an example of a complex problem he/she has faced and demonstrate students how this problem can be decomposed to find solution.

Hands-On-Practice: Each student should identify a complex problem that they have faced and prepare solution of this problem by decomposing the problem.

3.5 Algorithmic Thinking

Algorithmic thinking is a way of solving problems by producing algorithms. It is a method for solving problems based on a clear definition of the steps; logically and repeatedly.

3.5.1 Algorithm

- An algorithm is an ordered clear sequence of steps to solve a problem or accomplish a task.
- An algorithm is a plan that tells you how to do a task.
- We use algorithms all the time. for example, consider the steps that you take to
 - Prepare a pancake mix
 - Make a cup of tea
 - Convert temperature readings from Celsius to Fahrenheit



Figure 3.13 Making of a cup of tea

To develop an algorithm, we need to represent the instructions in some way that is understandable to a person who is trying to figure out the steps involved.

Two commonly used representations for an algorithm are by using.

- pseudocode,
- flow charts.

3.5.2 Pseudocode

Pseudocode is a description of algorithms using everyday language, similar to English, but structured to resemble a simplified programming language.

Consider the example of adding two numbers.

Pseudocode for adding two numbers

(Addition of two numbers) This algorithm performs addition of two numbers. The variable **A** and **B** will store the two numbers, while the variable **Sum** stores the sum of these two variables.

Step 1: Start

Step 2: Input two numbers A and B

Step 3: $\text{Sum} = A + B$

Step 4: Print the value of Sum.

Step 5: Stop

3.5.3 Flowchart

Flowcharts are diagrammatic representations that show a sequence of actions and the order in which they occur. For example, consider the example of adding two numbers.

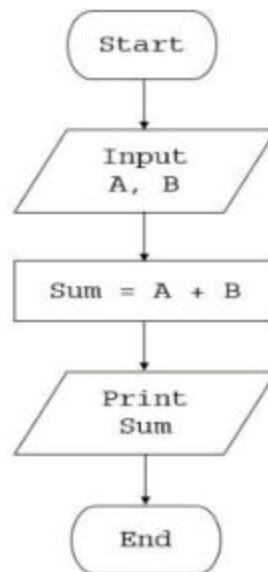


Figure 3.14 The flow chart of adding two numbers

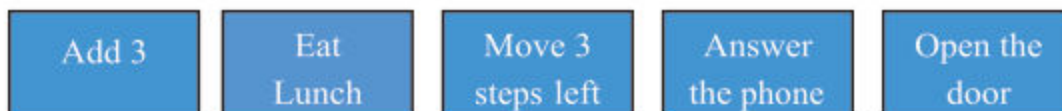
3.5.4 Symbols used in Flow Charts

The following shapes and symbols are used in flowcharts:

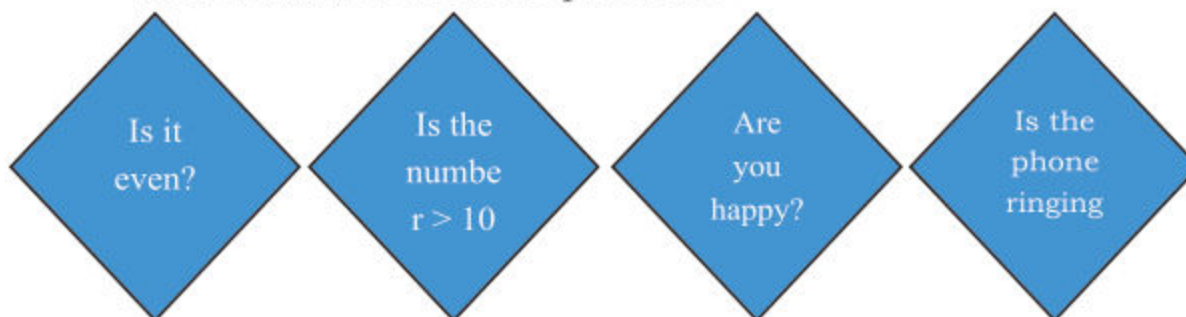
1. **Ovals** are used to show the beginning and end.



2. **Rectangles** are used to show actions.



3. **Diamonds** are used to indicate decisions that need to be made. Think of them as questions.



4. **Parallelograms** are used to indicate the input or output operations in your flowchart.



5. **Arrows** indicate the direction in which the process is flowing from one step to the next.



3.5.5 Guidelines for developing flowcharts.

- Flowcharts can only have one start and stop symbol.
- The general flow of processes (Arrows) is top to bottom or left to right.
- Arrows should not cross each other.
- In case of loops, the flow of process could be from bottom to top.

Example # 1: Calculating the cube of a given number

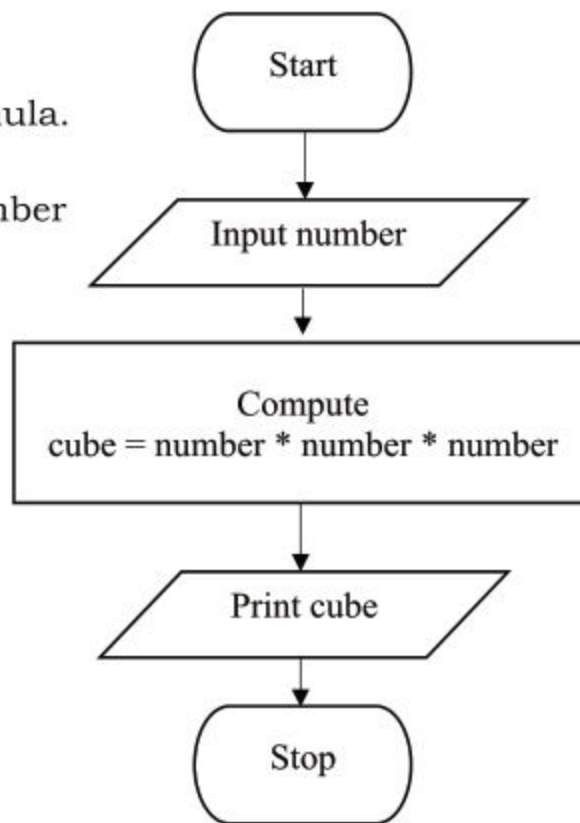
Pseudocode of finding cube of a given number

(Finding cube of a number) This algorithm takes a number as input from the user and then calculates its cube. The variable number stores the input number, and the cube stores the cube value calculated in the algorithm.

Step 1: Start
 Step 2: Input number
 Step 3: Compute cube using formula.

$\text{cube} = \text{number} * \text{number} * \text{number}$

Step 3: Print cube
 Step 4: Stop



3.5.6 Algorithm building blocks

An algorithm is made up of three building blocks.

- (I) **Sequencing:** The process must be executed in the same written order. [See Previous examples].
- (II) **Selection / Conditional:** The process executes different sets of steps based on a conditional statement. Steps are executed if the condition is true otherwise the next step is executed. For example, determining the largest number among all the entered numbers.

Pseudocode for finding the largest number between two given numbers

(Finding the largest number between two numbers). This algorithm will print the largest number between the two numbers

entered by the user. The variables A and B store the given numbers.

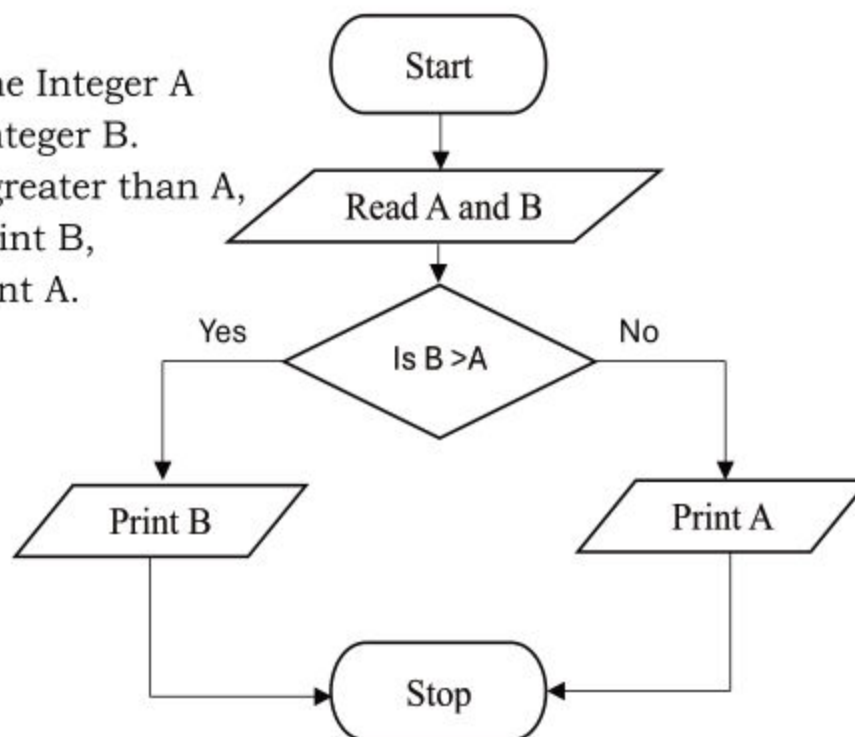
Step 1: Start

Step 2: Read the Integer A

Step 3: Read Integer B.

Step 4: If B is greater than A,
then print B,
else print A.

Step 5: Stop



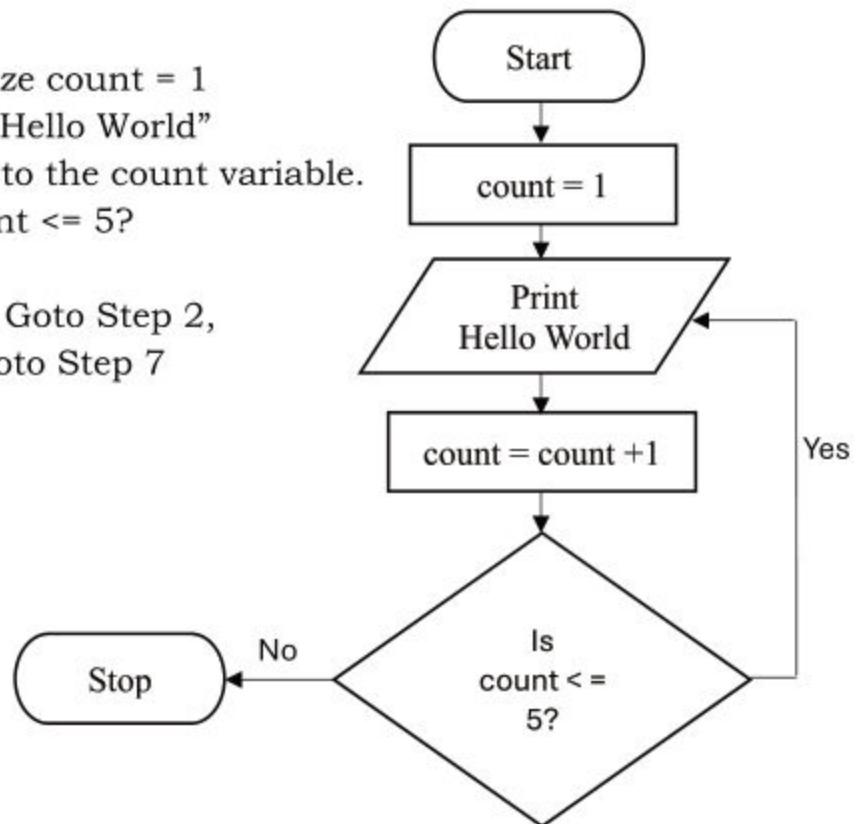
(III) **Repetitive / Iterative:** Repetitive flow, also referred to as iterative flow, is a process in which a statement or a set of statements is executed repeatedly till a certain condition remains true.

For example, Print “Hello World” five times. Here, we want to print the same sentence repeatedly, therefore iterative / repetitive process will be used here. A counter variable is needed to keep track of the repetition.

Pseudocode for printing Hello World on screen five times

This algorithm will print a simple message such as “Hello World” five times. The variable **count** acts as the counter variable.

Step 1: Start
 Step 2: Initialize count = 1
 Step 3: Print "Hello World"
 Step 4: Add 1 to the count variable.
 Step 5: Is count <= 5?
 Step 6: If Yes
 then Goto Step 2,
 else Goto Step 7
 Step 7: Stop



ACTIVITY

Teacher will engage the students to practice creating pseudocode and flowcharts for various problems such as:

1. Designing an algorithm for getting ready for school.
2. Designing an algorithm for planting a flower in the pot.
3. Designing an algorithm to determine if the given number is even or odd.
4. Designing an algorithm to find sum and average of three numbers.
5. Designing an algorithm for multiplying any two given numbers.
6. Designing an algorithm that take a number for user, it will print Car if the number is 1 otherwise print Bus.

Summary

- Problems are like challenges that make our brain grow stronger.
- Simple problems are like small puzzles waiting to be solved.

- Complex problems are like big adventures that needed more steps to solve.
- Decomposing a problem helps us understand the problem better.
- When solving problems,
 - First, we think about what we know (our facts), like ingredients of a recipe.
 - Next, we look what we start with (our inputs), like the tools for making a craft.
 - Then, we imagine what we want in the end (our output), such as finished drawing or solved puzzle.
- When faced with a problem, we need a plan to solve the problem known as algorithm.
- Algorithms are step by step recipes for solving problems.
- Algorithmic thinking is like having the superpower to solve problems.
- Flow charts are like visual map showing the steps of an algorithm.
- Flowcharts use symbols and arrows to guide us through the process.
- Pseudocode is a simple way to write down algorithms using everyday language just like writing a recipe in words before making a dish.

Important Terms to Remember

Problem	A problem is something (event, task, situation) that needs to be solved. It is like a puzzle or question for which a solution must be found.
Problem solving	The process of finding solutions to challenges or puzzles using logical thinking and creativity.
Problem decomposition	A problem-solving methodology that involves breaking down large problems into smaller, more manageable parts.
Iteration	Iteration in algorithms refers to the repetition of a process or set of instructions until a certain condition is met or a desired result is achieved.
Condition	A condition in algorithms is a logical statement or expression that determines the execution or repetition of certain instructions based on whether it evaluates to true or false.

Exercise

1. Encircle the correct answer

- i. The problem is a:
 - a. Toy
 - b. Challenge waiting to be solved.
 - c. Storybook
 - d. Game
- ii. The example of a simple problem is:
 - a. Building a robot
 - b. Tying your shoes
 - c. Creating a video game
 - d. Baking a cake

- iii. To create a problem-solving plan, we need:
 - a. Ingredients
 - b. Facts, Inputs, Outputs
 - c. Tools
 - d. details
- iv. Decomposing a problem means:
 - a. Making it bigger
 - b. Breaking it into smaller parts
 - c. Solving it quickly
 - d. Ignoring it
- v. Algorithmic thinking is like:
 - a. Being a detective solving a mystery
 - b. Being lost in a maze
 - c. Watching TV
 - d. Playing video games

2. Fill in the blanks with appropriate words given below.

Plan, language, part, small, process.

- (i) Simple problems are like _____ puzzles.
- (ii) Decomposing a problem helps us understand each _____ better.
- (iii) An algorithm is a step-by-step _____ for solving a problem.
- (iv) Pseudo code is a simple way to write down algorithms using every day _____.
- (v) Flow charts use symbols and arrows to guide us through a _____.

3. Provide descriptive answers to the following questions.

- (i). Give an example of a simple problem you've faced and how you solved it.
- (ii). Why is it important to break down a problem into smaller parts?
- (iii). List down the benefits of step-by-step process of problem solving.
- (iv). Can you think of a real-life situation where you might use an algorithm, like a step-by-step plan?
- (v). How does planning a solution reduce the risk of failure?

4. Class Activities

- (i). Solve the given scenario by following six step problem solving guideline. "Ali is having severe headache. He must consult a doctor for treatment."
- (ii). Apply the concept of problem decomposition and write down the sub-problems for the following scenario.

"Your mother asks you for grocery shopping and you need to go to the grocery shop".

The problem identified here is "How to do grocery shopping".
- (iii). Write an algorithm that allows the person to enter a number, algorithm will print table of that number up to 10.

- (iv). Write an algorithm that determines whether a student has passed the exam or not. The algorithm takes grades of four subjects as input and then the algorithm calculates the percentage obtained by the student. If the percentage of marks obtained by the student is greater than 60 % then the student has passed the exam otherwise student has failed the exam.

Hint. Simple decision making is needed to solve this problem.

- (v). Write an algorithm that calculates the sum of the first 5 numbers i.e. the sum of 1,2,3,4 and 5.

Hint: Repetitive/iterative flow statements are needed to solve this problem.

Note for Teachers

- Teachers must provide real-world problem-solving scenarios and challenge students to apply the techniques of problem solving they have learned.
- Teachers must explore practical applications of algorithmic thinking and problem-solving skills in various fields, such as computer science, engineering, mathematics, and everyday life.
- Engage students in discussions about the importance of breaking down complex problems into smaller, more manageable steps.
- Introduce the concept of algorithmic thinking. Explain that algorithms are step-by-step procedures used to solve problems.
- Use real-life examples to illustrate algorithms, such as recipes, instructions for assembling furniture, or navigating a route on a map.
- Use interactive demonstrations or group activities to reinforce these concepts.
- Teachers must teach students how to design algorithms by breaking down problems into smaller steps, identifying patterns, and developing clear and concise instructions.
- Guide students through the process of refining and improving their algorithms through testing and iteration.